

EXPERIMENT-1: TRAINING A ML MODEL ON TITANIC DATASET . USING GRID SEARCH AND RANDOM SAERCH PERFORM THE HYPERPARAMETER TUNING AND FIND OUT THE OPTIMAL HYPERPARAMETERS. COMPARE THE TUNED MODEL PERFORMANCE

```
In [3]: #1 Import Libraries
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
In [4]: #2 Load Dataset
df = pd.read_csv("titanic.csv")

print("Shape:", df.shape)
df.head()
```

Shape: (891, 12)

Out[4]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```
In [5]: #3 Data Preprocessing

# 3.1 Drop Irrelevant Columns
df.drop(["PassengerId", "Name", "Ticket", "Cabin"], axis=1, inplace=True)

#3.2 Handle Missing Values
# Fill Age with median
df["Age"].fillna(df["Age"].median(), inplace=True)

# Fill Embarked with mode
df["Embarked"].fillna(df["Embarked"].mode()[0], inplace=True)

#3.3 Encode Categorical Variables
#Columns needing encoding: Sex, Embarked, Pclass (categorical though numeric)

df = pd.get_dummies(df, columns=["Sex", "Embarked", "Pclass"], drop_first=True)

print("Processed Shape:", df.shape)
```

Processed Shape: (891, 10)

```
In [6]: #4. Define Features & Target
X = df.drop("Survived", axis=1)
y = df["Survived"]
```

```
In [7]: #5. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y #ensures all classes are equally distributed
)
```

```
In [8]: #6 Baseline Random Forest
rf = RandomForestClassifier(random_state=42)

rf.fit(X_train, y_train)

y_pred_baseline = rf.predict(X_test)

baseline_acc = accuracy_score(y_test, y_pred_baseline)

print("Baseline Accuracy:", baseline_acc)
print(classification_report(y_test, y_pred_baseline))
```

Baseline Accuracy: 0.8156424581005587

	precision	recall	f1-score	support
0	0.83	0.88	0.85	110
1	0.79	0.71	0.75	69

accuracy			0.82	179
macro avg	0.81	0.80	0.80	179
weighted avg	0.81	0.82	0.81	179

```
In [9]: #7 Grid Search
        #Define Parameter Grid
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5]
}

        #Apply GridSearchCV
grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("Best Parameters (Grid):", grid_search.best_params_)
```

Best Parameters (Grid): {'max_depth': 5, 'min_samples_split': 2, 'n_estimators': 100}

```
In [10]: #8 Evaluate Grid Model
grid_model = grid_search.best_estimator_

y_pred_grid = grid_model.predict(X_test)

grid_acc = accuracy_score(y_test, y_pred_grid)

print("Grid Search Accuracy:", grid_acc)
```

Grid Search Accuracy: 0.8100558659217877

```
In [11]: #10 Random Search
        #Define Parameter Distribution
param_dist = {
    'n_estimators': np.arange(50, 300, 50),
    'max_depth': [None, 5, 10, 15],
    'min_samples_split': [2, 5, 10]
}

        #Apply RandomizedSearchCV
random_search = RandomizedSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_distributions=param_dist,
    n_iter=10,
    cv=5,
    scoring='accuracy',
    random_state=42,
    n_jobs=-1
)

random_search.fit(X_train, y_train)

print("Best Parameters (Random):", random_search.best_params_)
```

Best Parameters (Random): {'n_estimators': 150, 'min_samples_split': 10, 'max_depth': None}

```
In [12]: #11 Evaluate Random Model
random_model = random_search.best_estimator_

y_pred_random = random_model.predict(X_test)

random_acc = accuracy_score(y_test, y_pred_random)

print("Random Search Accuracy:", random_acc)
```

Random Search Accuracy: 0.8044692737430168

```
In [13]: #12 Final Comparison
comparison = pd.DataFrame({
    "Model": ["Baseline", "Grid Search", "Random Search"],
    "Accuracy": [baseline_acc, grid_acc, random_acc]
})

print(comparison)
```

	Model	Accuracy
0	Baseline	0.815642
1	Grid Search	0.810056
2	Random Search	0.804469

Tuning might not bring major improvement because Default Random Forest Is Already Strong

1 Random Forest:

```
#Is already robust
#Works well with default parameters
#On Titanic (~891 rows), default settings are often already near optimal.
#So tuning might not bring major improvement.
```

2 With small datasets:

```
#Hyperparameter tuning via cross-validation may overfit to folds
#Best CV score ≠ Best test performance
```

3 If baseline model is slightly overfitting:

```
#Tuning may reduce overfitting → slightly lower test accuracy
#But actually better generalization.
#Lower accuracy ≠ worse model always.
```

In []: